

LLM-QL: A LLM-Enhanced Q-Learning Approach for Scheduling Multiple Parallel Drones

Qian Zhou , *Member, IEEE*, Jiayang Wu , Mengyue Zhu, Yuhang Zhou , Fu Xiao , *Senior Member, IEEE*, and Yanchun Zhang , *Member, IEEE*

Abstract—This study addresses the Multiple Flying Sidekicks Traveling Salesman Problem (mFSTSP), where parallel Unmanned Aerial Vehicles (UAVs), or Drones, work alongside truck to enhance delivery efficiency. Existing scheduling approaches face challenges in high computational costs and the risk of converging to local optima due to excessive exploration in unknown environments, especially in large-scale mFSTSP. This study proposed a Large Language Model Enhanced Q-Learning Approach (LLM-QL) to solve mFSTSP, which combines the local exploration advantages of Q-Learning with the global understanding of unknown environments provided by LLMs, thus improving the efficiency of path planning. A novel prompt strategy is also provided, transforming the problem modeling into a format easily understood by LLMs, guiding the algorithm's exploration and significantly improving convergence. We also provide a proof of the convergence of LLM-QL. Experimental results demonstrate that LLM-QL achieves up to a 1.35 x improvement in key performance metrics such as total completion time, algorithm runtime, and UAV utilization, compared to existing state-of-the-art methods.

Index Terms—Large language models, reinforcement learning, multiple flying sidekicks traveling salesman problem, unmanned aerial vehicle.

I. INTRODUCTION

UNMANNED Aerial Vehicles (UAVs), also referred to as drones, have been increasingly integrated into last-mile delivery scenarios alongside trucks [1], significantly reducing the delivery time compared to using a truck alone [2], [3]. This multiple parallel drones delivery model is known as the Flying Sidekick Traveling Salesman Problem (FSTSP), which was first introduced by Murray in 2015 [4].

Received 8 January 2025; revised 14 April 2025; accepted 10 June 2025. Date of publication 12 June 2025; date of current version 24 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62372244 and Grant 61902199, in part by the Major Scientific Instruments and Equipments Development Project of National Natural Science Foundation of China under Grant 62427809, in part by National Key Research and Development Program of China under Grant 2024YFB2906700, and in part by the Key Program of Natural Science Foundation of Jiangsu under Grant BK20243053. Recommended for acceptance by Y. Tong. (*Corresponding author: Qian Zhou.*)

Qian Zhou and Fu Xiao are with the School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: zhouqian@njupt.edu.cn; xiaof@njupt.edu.cn).

Jiayang Wu, Mengyue Zhu, and Yuhang Zhou are with the School of Modern Posts, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: 1223096926@njupt.edu.cn; 1223096927@njupt.edu.cn; 1224097338@njupt.edu.cn).

Yanchun Zhang is with Zhejiang Normal University, Jinhua 321004, China, and also with the Institute for Sustainable Industries and Liveable Cities, Victoria University, Melbourne, VIC 3011, Australia (e-mail: yanchun.zhang@vu.edu.au).

Digital Object Identifier 10.1109/TKDE.2025.3579386

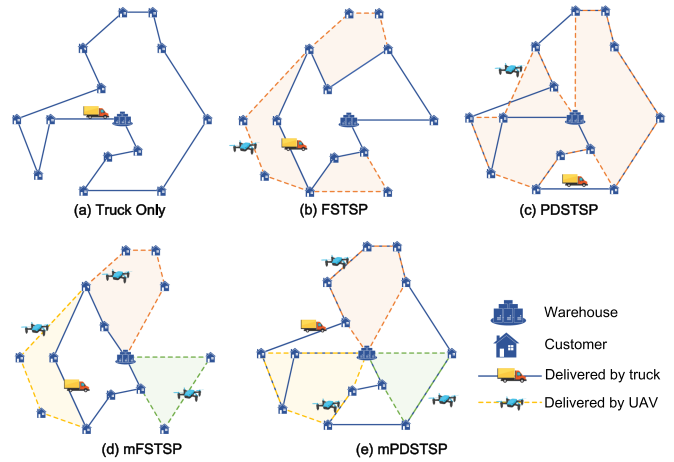


Fig. 1. Classification of truck and UAVs delivery scenarios.

Building upon the FSTSP, researchers have proposed various extensions and variants [5], [6], [7], [8], [9]. Following Murray's naming convention, existing research can be classified into four categories based on the number of UAVs and their relationship with trucks, as shown in Fig. 1: “FSTSP (Fig. 1(b)) [4], [7]” “Parallel Drone Scheduling TSP (PDSTSP)(Fig. 1(c)) [10]” “Multiple Flying Sidekicks TSP (mFSTSP) (Fig. 1(d)) [2]”, and “Multiple Parallel Drones Scheduling TSP (mPDSTSP) (Fig. 1(e)) [11]”. For comparison, the situation without UAVs is shown in Fig. 1(a).

In the FSTSP and mFSTSP, the truck's path must form a closed loop, with UAV(s) taking off and landing from the truck, which serves as both a mobile warehouse and transportation resource. The truck can transport UAVs closer to customer locations for takeoff and landing, as well as recharge UAVs during transport when no flying tasks are in progress. In contrast, for the PDSTSP and mPDSTSP, UAV(s) can only take off from a centralized warehouse and serve nearby customers. The mFSTSP eliminates the need for UAVs to consider flight time constraints by returning to a centralized warehouse for recharging and cargo replenishment [12], [13]. Additionally, it reduces the need for trucks to serve hard-to-reach customers, improving overall delivery efficiency and flexibility, reducing energy consumption, and shortening delivery times, which has made it a popular area of research [14], [15], [16].

In the mFSTSP scenario, the complexity of path planning and the difficulty of coordination between UAVs and the

truck increase significantly [17]. The key difference between single and multiple UAV scenarios lies in the coordination and management of multiple UAVs to optimize delivery efficiency. In single UAV scenarios, the truck is solely responsible for UAV coordination, but in multiple UAV scenarios, additional challenges arise in managing simultaneous flights and ensuring minimal interference. To address these challenges, most existing methods employ phased and hierarchical approaches for scheduling, utilizing various deep learning, reinforcement learning, or heuristic methods at different stages and levels for planning and scheduling [2]. Jeong et al. [18] proposed a two-stage construction and search heuristic algorithm focused on the hybrid delivery system of trucks and UAVs. This algorithm optimizes the path considering factors such as UAV energy consumption, package weight, and no-fly zones, offering high practical applicability. However, there is still room for improvement in managing path conflicts and integrating real-time data. Despite these advancements, these methods continue to face challenges in large-scale delivery tasks, such as high computational resource consumption, low exploration efficiency, and a tendency to converge to local optima, due to the lack of global environmental understanding [19].

In recent years, the application of Large Language Models (LLMs) in guiding heuristic algorithms, reinforcement learning algorithms, and other optimization methods has achieved remarkable success, demonstrating the superior contextual understanding and global reasoning capabilities of LLMs [20], [21], [22]. In problems such as combinatorial optimization, path planning, and game theory, LLM-assisted algorithms quickly generate reasonable heuristic rules, intermediate variables, or adaptive optimization strategies, accelerating the problem-solving process and avoiding local optima [23], [24], [25]. The Q-Learning algorithm, as a classical reinforcement learning technique, is widely used to solve optimal strategy problems in decision-making processes, such as path planning. Its core idea is to learn an action-value function to estimate the expected return of taking an action in a given state, thereby guiding the agent to select the optimal strategy [26], [27]. Q-Learning can suffer from excessive ineffective exploration when dealing with high-dimensional and large-scale problems, often leading to incomplete environmental understanding [28], [29], [30]. However, the global environmental understanding provided by LLMs can help guide Q-Learning towards faster convergence, though only a few studies, such as [31], have explored this direction.

Based on the aforementioned issues, the contributions of this study are as follows:

- 1) A LLM-Enhanced Q-Learning approach (LLM-QL) is proposed to solve mFSTSP. By leveraging the global environmental understanding of LLMs and the local exploration advantages of Q-Learning, heuristic items are generated to guide the agent's exploration, significantly reducing ineffective explorations and achieving faster convergence.
- 2) A prompt strategy tailored for mFSTSP is designed, transforming the modeling of the problem into a format that is easier for the LLM to understand. Additionally, the convergence of the Q-Learning algorithm with LLM-generated heuristic items is demonstrated.

- 3) Simulation experiments conducted on a real urban dataset from Seattle show that LLM-QL achieves up to a 1.35 x improvement in key performance metrics, such as total completion time, algorithm runtime, and UAV utilization, compared to existing advanced methods.

II. RELATED WORK

A. Multiple Flying Sidekicks Traveling Salesman Problem

Murray et al. [2] first proposed mFSTSP, which is solved by a three-phase iterative heuristic algorithm. This approach first determines the truck route, divides customers into truck-served and UAV-served groups, and further optimizes service time using mixed-integer linear programming. This algorithm provides optimal solutions within a reasonable timeframe and is suitable when UAV flight ranges are limited. However, its computational complexity is high, and it becomes inefficient in larger-scale scenarios. Moshref et al. [32] introduced an efficient truck and UAV route planning algorithm, where the truck acts as a mobile warehouse to enable multi-modal delivery in collaboration with UAVs. This method generates an initial solution using simulated annealing and refines it through heuristic adjustments, achieving high search efficiency and feasibility. However, its performance is limited in dynamic multi-UAV scheduling. Sacramento et al. [33] designed an adaptive large-range search meta-heuristic to address the UAV-collaborative vehicle routing problem, optimizing cost under time and capacity constraints. This approach is efficient in path planning tasks and balances search time and accuracy in most scenarios, but it is difficult to extend to dynamic multi-UAV scenarios. Additionally, Ham et al. [10] proposed a constraint programming-based system integrating multiple trucks and UAVs, suitable for complex scenarios with time windows and task priorities. By considering task dependencies and sequence in path selection, the system provides more flexible and applicable delivery solutions. However, the algorithm has high computational resource demands, resulting in lower efficiency in large-scale UAV collaborative scheduling. Arishi et al. [34] introduced a two-stage machine learning method combining constraint clustering and deep reinforcement learning. In the first stage, k-means clustering is applied to customer locations, and in the second stage, deep reinforcement learning is used to find the optimal path within each constraint cluster. Bi et al. [35] proposed a multi-agent reinforcement learning approach, using Monte Carlo tree search to break the vehicle routing problem into stages, accelerating truck route determination, and employing multi-agent reinforcement learning to solve UAV flight trajectories, enhancing learning efficiency and stability through regularization techniques.

As shown in Table I, existing methods for addressing large-scale delivery tasks and dynamic environments still face issues such as high computational resource consumption, low exploration efficiency, and a tendency to converge to local optima.

B. Q-Learning Method

Q-Learning, as a key reinforcement learning method, is widely used in path planning and UAV scheduling tasks. Yan et al. [36] proposed a Q-Learning-based UAV path planning

TABLE I
SUMMARY OF RELATED WORK

Category	Ref.	CD	MUS	RL	HA	RTF
mFSTSP	[4]	✓	-	-	✓	-
	[33]	✓	✓	-	✓	-
	[34]	✓	-	-	✓	-
	[11]	✓	✓	-	-	-
	[35]	✓	✓	✓	-	-
	[36]	✓	✓	✓	-	✓
Q-Learning	[37]	-	✓	✓	-	-
	[38]	-	-	✓	-	-
	[39]	-	✓	✓	-	-
	[40]	-	✓	✓	-	-
LLM	[41]	-	-	-	✓	-
	[42]	-	-	-	✓	-
	[43]	-	-	-	-	✓
	[44]	-	-	-	✓	✓
	[32]	-	-	✓	✓	-

Definitions of Acronyms: CD: Coordinate Delivery; MUS: Multi-UAV Scheduling; RL: Reinforcement Learning; HA: Heuristic Approach; RTF: Real-Time Feasibility.

algorithm for hostile environments, setting up a grid-based environment and designing a reward system to help UAVs avoid dangerous areas. This method exhibits high robustness in complex environments but suffers from low learning efficiency, making it difficult to coordinate path selection in multi-UAV scenarios. Li et al. [37] introduced a hybrid A* and Q-Learning algorithm to enhance the precision of UAV path planning. By improving A*'s search strategy and Q-Learning's exploration factor, the method boosts path planning efficiency but is sensitive to parameters, requiring fine-tuning in complex tasks to maintain stable performance. Wu et al. [38] proposed an adaptive velocity Q-Learning algorithm for UAV search and rescue tasks in unknown environments. By dynamically adjusting the UAV's speed, the method improves exploration efficiency and optimizes path selection through sub-domain search, enabling faster adaptation to complex environments. However, its computational cost is high, and its efficiency is limited in large-scale problems. Beishenalieva et al. [39] proposed a Q-Learning algorithm based on spatiotemporal sub-states, optimizing UAV data collection paths in sensor networks while satisfying data value maximization and energy consumption constraints. This algorithm enhances UAV data collection efficiency, making it suitable for dynamic data acquisition in sensor networks, but its real-time performance in large-scale applications still needs improvement.

As shown in Table I, when applying Q-Learning in large-scale scenarios, existing methods encounter problems such as low learning efficiency, high computational costs, parameter sensitivity, and poor real-time performance.

C. Large Language Model Enhancement Strategy

The development of LLMs has provided enhanced strategies for heuristic methods, reinforcement learning, and other optimization techniques in path planning. Luo et al. [40] explored the application of LLMs in multimodal data, proposing the

“Valley” model, which combines a visual encoder and a temporal modeling module to enhance UAV environmental perception in path planning. This method performs excellently in complex scenarios, but its high computational complexity due to the integration of multiple components makes it unsuitable for real-time path planning. Zhu et al. [41] proposed MiniGPT-4, which effectively aligns visual features with language models to achieve multimodal generative functions. Although this model provides high-quality environmental information for path planning, its inference speed is limited by the high complexity of the model, restricting its application in real-time scenarios. Trajanoska et al. [42] constructed a knowledge graph using LLMs to improve the accuracy of information extraction in path planning. This method enhances the efficiency of large-scale information processing but still requires further improvement in real-time performance optimization for path planning tasks. Meng et al. [43] introduced the LLM-A algorithm, which combines the path-searching capability of A* with the global reasoning ability of LLMs, significantly improving path planning efficiency in large-scale scenarios. The advantage of this method lies in optimizing computation time and memory usage, making it suitable for large-scale path planning. However, it has limitations in real-time task responsiveness. Wu [31] proposed an LLM-guided Q-Learning framework, incorporating LLM-generated heuristic values into Q-function learning, enabling more efficient path planning in complex environments. The advantage of this method is its improved sampling efficiency and reduced ineffective exploration. However, due to the high computational demands of LLMs, it increases the overall computational burden in large-scale environments.

As shown in Table I, existing methods that use LLMs to guide optimization algorithms mainly face issues such as hallucination problems and insufficient precision in calculations, leading to slow model inference and high computational complexity, which limits their effectiveness in real-time path planning applications.

III. LLM-ENHANCED Q-LEARNING APPROACH

A. Problem Formulation

In mFSTSP, UAVs are launched from the truck to deliver packages to specific customers, then return to the truck for recharging. The truck and UAVs work in coordination to minimize the total delivery time. This problem is applicable to complex urban logistics, especially in scenarios where truck delivery times are long, and UAVs can complete deliveries quickly. The symbols involved in this problem are listed in Table II. A critical aspect of the coordination between UAVs and trucks is the waiting process that ensures efficient synchronization. Specifically, the truck must remain stationary during the UAV's launch and recovery phases, preventing the truck from moving while the UAV is in transit to and from the designated customer locations. Similarly, the UAV needs to wait for the truck to reach a designated location before it can safely land and recharge.

The overall objective of mFSTSP is to minimize the completion time of the delivery task, i.e., the time taken for the truck and all UAVs to complete their tasks and return to the starting

TABLE II
PRIMARY NOTATIONS

Symbols	Meaning
V	Set of UAVs
C	Set of customers, each requiring one parcel delivery
C_v	Set of customers that can be served by UAV v
N	Set of nodes, including warehouse and customers
N_0	Set of nodes excluding the ending depot node
c	Number of customers ($ C $)
τ_{ij}	Travel time of the truck from node i to node j
τ_{vij}	Flight time of UAV v from node i to node j
e_{vijk}	Maximum endurance time of UAV v to fly from i to j and return to k
x_{ij}	Binary variable, $x_{ij} = 1$ if the truck travels from node i to node j , otherwise $x_{ij} = 0$
y_{vijk}	Binary variable, $y_{vijk} = 1$ if UAV v departs from truck at node i , visits customer j , and returns to truck at node k ; otherwise $y_{vijk} = 0$
t_i	Arrival time of the truck at node i
u_i	Auxiliary variable used to eliminate subtours for node i

point:

$$\min t_{c+1} \quad (1)$$

According to the literature [18], [19], [32], [33], [44], the problem can be subjected to the following constraints:

$$\sum_{i \in N_0, i \neq j} x_{ij} + \sum_{v \in V} \sum_{i \in N_0, i \neq j} \sum_{k \in N} y_{vijk} = 1, \quad \forall j \in C \quad (2)$$

$$\sum_{j \in N} x_{0j} = 1, \quad \sum_{i \in N} x_{i,c+1} = 1 \quad (3)$$

$$\sum_{j \in C, j \neq i} \sum_{k \in N} y_{vijk} \leq 1, \quad \forall i \in N_0, \forall v \in V \quad (4)$$

$$\tau_{vij} + \tau_{vjk} \leq e_{vijk}, \quad \forall v \in V, \forall i, j, k \in N \quad (5)$$

$$t_j \geq t_i + \tau_{ij} - 1 - x_{ij}, \quad \forall i, j \in N, i \neq j \quad (6)$$

$$\sum_{i \in N, i \neq j} x_{ij} = \sum_{k \in N, k \neq j} x_{jk}, \quad \forall j \in C \quad (7)$$

$$y_{vijk} \Rightarrow x_{ik} = 1, \quad \forall v \in V, i \in N_0, j \in C, k \in N \quad (8)$$

$$i \neq k, \quad \forall v \in V, i, k \in N, j \in C \quad (9)$$

$$t_j \leq t_i + e_{vijk}, \quad \forall i, j, k \in N, \forall v \in V \quad (10)$$

$$u_i - u_j + (c + 2)x_{ij} \leq c + 1, \quad \forall i, j \in C, i \neq j \quad (11)$$

Equation (2) represents the delivery constraint for customers, ensuring that each customer j is visited only once, either by the truck (through x_{ij}) or by one UAV v (through y_{vijk}). Equation (3) ensures that the truck starts from the depot (node 0) and ends at node $c + 1$. Equation (4) limits each UAV v to serve at most one customer j per task. Equation (5) ensures the total flight time of UAV v does not exceed its endurance limit e_{vijk} . Equation (6) updates the truck's arrival time t_j based on its travel from node i to node j , depending on x_{ij} . Equation (7) ensures the truck's in-degree and out-degree are equal at customer node j , maintaining path connectivity. Equation (8) ensures that if UAV v is dispatched from i to serve j and return to k , then the truck must also move from i to k ($x_{ik} = 1$). Equation (9)

prevents UAVs from launching and returning to the same node. Equation (10) ensures UAV v finishes service at node j within its endurance window. Equation (11) introduces auxiliary variables u_i to eliminate subtours by enforcing ordering constraints among nodes.

The mFSTSP can be viewed as a sequential decision-making problem. Each decision consists of two elements: the current location of the truck and the chosen delivery mode (truck or UAV). Q-learning is suitable for combinatorial optimization problems with large state and action spaces. Based on the current delivery state, it selects the next suitable action from multiple customers and tools. The agent can gradually explore and learn the optimal collaborative strategy between the truck and UAVs to minimize the total delivery time. It can also dynamically adapt to complex constraints, progressively converging to the optimal solution during iterations. Solving mFSTSP using Q-learning requires defining the state, action, and Q-table.

Each state $S = (i, D)$ can be represented as a combination of the current delivery task's specific location and the UAV's status. Here, i represents the current node where the truck is located, and D indicates the UAV's status, recording whether the UAV is in the task. If the UAV is deployed, its current position is also recorded.

Each action $A = (j, m)$ represents the path selection from the current node to the next target node and the choice of transport mode. Here, j denotes the next target node, and m represents the choice of transport tool, where 0 indicates using the truck and 1 indicates using a UAV. The action space includes the set of unvisited nodes and the choice between two transport modes. When selecting an action, constraints from (2) to (11) need to be considered.

The Q-table $Q[i, j, m]$ is a three-dimensional array used to store the expected rewards for taking different actions in different states. Each entry in the Q-table represents the cumulative reward gained from traveling from node i to node j using tool m . Therefore, the first dimension of the Q-table corresponds to the current truck node, the second dimension corresponds to the next node, and the third dimension corresponds to the transport mode, where 0 represents the truck and 1 represents the UAV. The Q-value update formula is:

$$Q[i, j, m] = Q[i, j, m] + \alpha \left(R[i, j, m] + \gamma \max_{A'} Q[j, k, m'] \right) - Q[i, j, m] \quad (12)$$

Where α represents the learning rate, which controls the impact of new information on the existing Q-values, while γ denotes the discount factor, reflecting the importance of future rewards. The reward function $R[i, j, m]$ is defined as the reciprocal of the path distance from node i to node j , with either the truck or UAV being used. A shorter distance results in a higher reward, incentivizing the learning algorithm to select shorter paths. The mFSTSP is reformulated as a sequential decision-making problem, which can be addressed through Q-learning. By learning and updating the Q-table, the optimal selection of transport tools and paths can be determined.

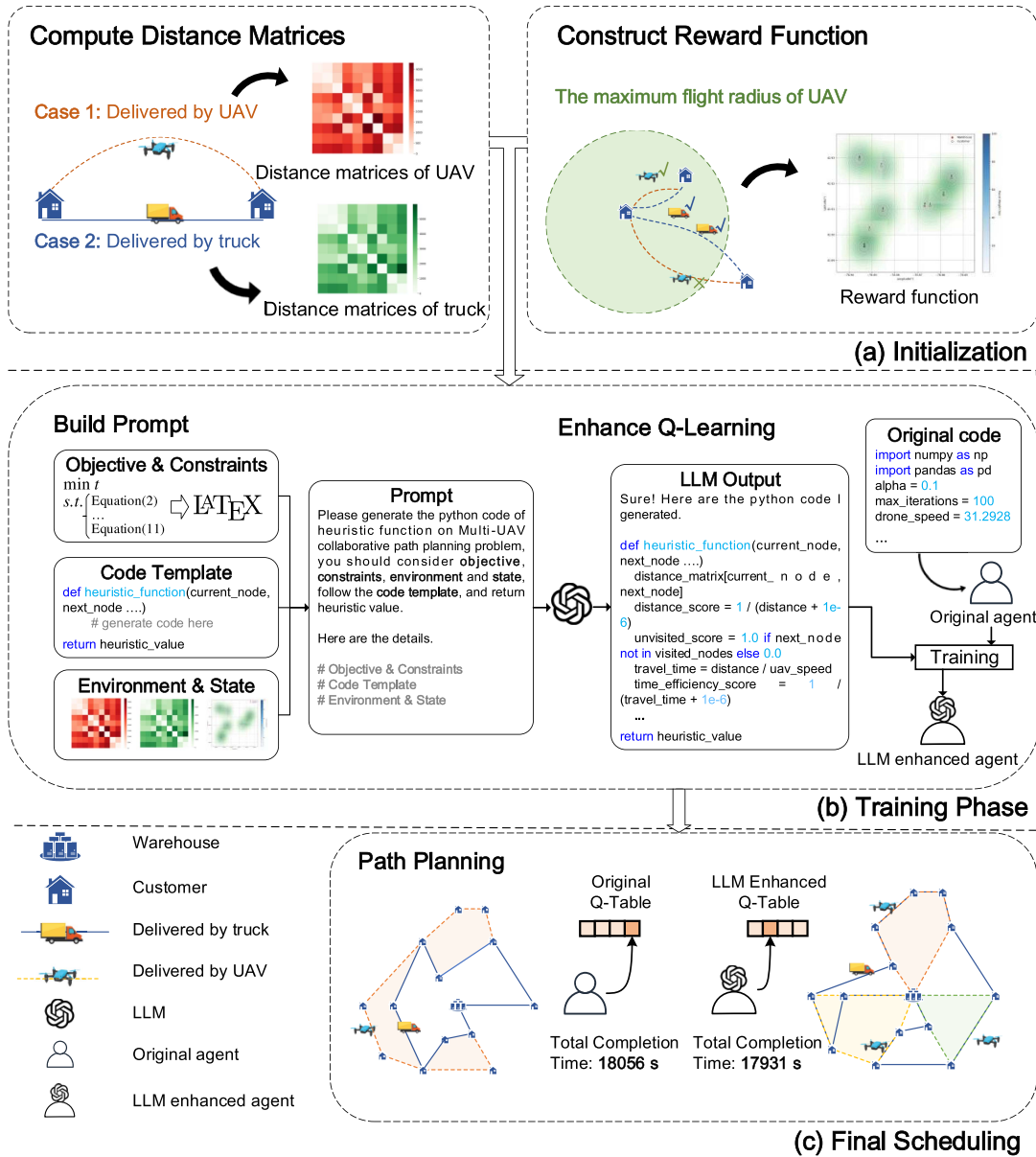


Fig. 2. Overall framework of LLM-QL.

B. Using Large Language Models to Enhance Q-Learning for Optimization Problems

Traditional Q-Learning methods face challenges such as low learning efficiency, slow convergence, and local optima when solving mFSTSP due to the high-dimensional state space and complex constraints. LLM-QL introduces LLMs into the Q-Learning framework to generate heuristic information that guides exploration, significantly accelerating the algorithm's convergence process. LLM-QL enhances the efficiency and robustness of Q-Learning by designing a well-structured reward function, developing prompt strategies tailored to the problem's characteristics, and addressing potential hallucination phenomena that may arise from the LLM. The overall framework of LLM-QL is shown in Fig. 2.

During the initialization phase (Fig. 2(a)), two distinct distance matrices are calculated to represent the different delivery modes: by truck and UAV. For truck deliveries, the actual road network distance between points is used to reflect realistic routing constraints, while UAV deliveries are modeled using euclidean distances, assuming direct flight paths between locations. These distance matrices serve as the foundation for constructing a reward function, where the reward is higher for shorter transportation times. This reward function is further refined by incorporating the maximum flight radius of the UAVs to ensure that the delivery plans remain feasible within operational limits. In the training phase (Fig. 2(b)), the original agent is enhanced through the use of LLM. The LLM takes as input a set of parameters, including the objective function and constraints in the style of Latex, a Python code template, and the

Algorithm 1: Heuristic Item Generation Algorithm.

Require: Current node i , next node j , distance matrix D , priority weights P

Ensure: Heuristic value $H(i, j, m)$

- 1: Initialize heuristic value $H(i, j, m)$.
- 2: Calculate distance score: Assign a higher score for shorter distances.
- 3: Calculate unvisited score: Assign score 1 if node j is unvisited, else 0.
- 4: Get priority score: Use the value from priority weights $P[j]$.
- 5: Initialize connection score to 0.
- 6: **for** each node k in D **do**
- 7: **if** node k is unvisited and within threshold distance from node j **then**
- 8: Increment connection score.
- 9: **end if**
- 10: **end for**
- 11: Calculate time efficiency score: Assign a higher score for shorter travel time between i and j .
- 12: Combine scores with weights to compute final $H(i, j, m)$.
- 13: **return** $H(i, j, m)$

current environment and state, and generates a corresponding heuristic function in Python code. This heuristic function is then utilized by the Q-Learning algorithm to guide the LLM enhanced agent's action selection, improving decision-making by reducing ineffective exploration and computational overhead. The integration of the LLM ensures that the agent can make informed, optimized choices based on a global understanding of the environment, enhancing both the efficiency and effectiveness of the path planning process. Finally, in the final scheduling phase (Fig. 2(c)), the trained Q-Learning agent, now augmented with the LLM-generated heuristics, produces the optimal collaborative scheduling plan for the truck and multiple UAVs. This scheduling plan is illustrated by comparing the performance of the original Q-Table with that of the LLM-enhanced Q-Table. The results show that the LLM-enhanced Q-Table significantly reduces the total completion time of the mission, demonstrating a notable improvement in planning efficiency.

The reward function is a crucial element in reinforcement learning, guiding the agent's behavior. For the mFSTSP, the reward function needs to reflect both the efficiency of path planning and the rationality of resource utilization. The reward function in LLM-QL is designed as follows:

$$R[i, j, m] = \begin{cases} \frac{1}{T[i, j, m]} & \text{If all constraints are satisfied} \\ -\infty & \text{If any constraint is violated} \end{cases} \quad (13)$$

where $T[i, j, m]$ represents the time required to travel from node i to node j using the chosen transport tool m . Shorter travel times yield higher rewards. If any of the constraints from (2) to (11) are violated, the immediate reward is set to negative infinity, preventing the agent from selecting actions that are not feasible.

The prompt strategy involves inputting the problem modeling results into the LLM in the form of natural language or mathematical formulas to leverage its reasoning ability and global environmental awareness. The mFSTSP involves complex constraints and state transitions. To ensure the LLM correctly understands the problem state and possible action choices, the current state and relevant constraints are input in a formalized manner, reducing ambiguity inherent in natural language expression. Examples of heuristic item provided by ChatGPT-4o are presented in Algorithm 1.

Large language models can generate heuristic items $H[i, j, m]$ based on explicit mathematical formulas. As shown in Algorithm 1, the heuristic item generation process evaluates the attractiveness of a candidate node based on several factors. First, the algorithm computes a distance score, assigning a higher value for shorter distances between nodes. It then calculates an unvisited score, prioritizing nodes that have not yet been visited. The priority score, derived from predefined priority weights, reflects the importance of each node. Additionally, the algorithm evaluates the connection score by checking the proximity of other unvisited nodes to the current node. Finally, a time efficiency score is computed, favoring faster travel times between nodes. These heuristic items are executable Python code that, given the current state and possible actions, outputs a heuristic value influencing the Q-value update. Based on (12), the Q-value update formula with the heuristic item incorporated is as follows:

$$Q[i, j, m] \leftarrow Q[i, j, m] + \alpha \left(R[i, j, m] + \gamma \max_{A'} Q[j, k, m'] \right) + H[i, j, m] - Q[i, j, m] \quad (14)$$

As shown in Algorithm 2, LLM-generated heuristics to guide exploration improves traditional Q-Learning. In each episode, the algorithm selects actions based on an ϵ -greedy policy, computes a heuristic value $H[i, j, m]$ from the LLM, and updates the Q-values using (14). This heuristic enhances the agent's decision-making process, accelerating convergence and ensuring more efficient exploration. The process continues until all nodes are visited, and the optimal policy π^* is derived by selecting the action with the highest Q-value.

In LLMs, "hallucination" refers to the generation of overestimated or underestimated values [45]. In the context of Q-Learning, overestimation of Q-values can lead to overly large Q-values, while underestimation may cause important state-action pairs to be overlooked, thus affecting the strategy selection and overall convergence.

Assuming that the heuristic item $H[i, j, m]$ causes hallucination, leading to overestimation of Q-values, Theorem 2 in Section III-C proves that the impact of overestimation on the final solution is bounded within a certain limit $\frac{\lambda_H}{1-\gamma}$.

Underestimation can be classified into two scenarios: underestimation of the optimal action and underestimation of non-optimal actions. If non-optimal actions are underestimated, this does not affect the final strategy choice, as these actions are not optimal to begin with. However, underestimation of the optimal action can affect the strategy, as the optimal action may be mistakenly regarded as suboptimal and thus ignored.

Algorithm 2: Q-Learning Algorithm with Large Language Model Heuristics.

Require: Initialize $Q(i, j, m)$, step-size α , discount factor γ , exploration rate ε , heuristic function $H(i, j, m)$

Ensure: $\pi^* = \arg \max Q(i, j, m)$

```

1: for each episode do
2:   Initialize start node and visited nodes
3:   repeat
4:     Choose  $j$  and  $m$  from  $i$  using  $\varepsilon$ -greedy policy
       derived from  $Q$ 
5:     Take action  $(j, m)$ , observe  $R_{t+1}$  and next
       state
6:     Compute heuristic value in Algorithm 1
7:     Update Q-table using (14)
8:     Update visited nodes and set  $i \leftarrow j$ 
9:   until all nodes are visited and  $i = \text{start\_node}$ 
10: end for

```

To mitigate this, the Q-value update process involves more frequent sampling of high-reward paths, gradually increasing their Q-values to prevent the underestimation of truly optimal actions.

C. Convergence Proof

The Q-value function $Q[i, j, m]$ is used to estimate the cumulative reward obtained by taking an action (moving from node i to node j using vehicle m) in a given state. Through iterative updates, $Q[i, j, m]$ is expected to converge to the optimal Q-value function $Q^*[i, j, m]$, which guides the selection of the optimal policy.

According to (12), as the iteration progresses, $Q[i, j, m]$ should gradually approach the optimal value $Q^*[i, j, m]$, satisfying the Bellman optimality equation:

$$Q^*[i, j, m] = R[i, j, m] + \gamma \max_{A'} Q^*[j, k, m'] \quad (15)$$

To study the effect of the heuristic term, we define the traditional Bellman operator $\mathcal{T}$ and the heuristic-augmented Bellman operator $\mathcal{T}_H$ as follows:

$$\begin{cases} \mathcal{T}Q[i, j, m] = R[i, j, m] + \gamma \max_{A'} Q[j, k, m'] \\ \mathcal{T}_H Q[i, j, m] = R[i, j, m] + H[i, j, m] + \gamma \max_{A'} Q[j, k, m'] \end{cases} \quad (16)$$

For any two Q-value functions, Q_1 and Q_2 , the Bellman operator $\mathcal{T}$ satisfies the standard contraction property:

$$\|\mathcal{T}Q_1 - \mathcal{T}Q_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty \quad (17)$$

Here, $\|\cdot\|_\infty$ denotes the infinity norm, defined as:

$$\|Q_1 - Q_2\|_\infty = \max_{i,j,m} |Q_1[i, j, m] - Q_2[i, j, m]| \quad (18)$$

It can be proven that the traditional Bellman operator is a contraction mapping [46]:

$$\begin{aligned} & \|\mathcal{T}Q_1[i, j, m] - \mathcal{T}Q_2[i, j, m]\| \\ &= |\gamma \max_{A'} Q_1[j, k, m'] - \gamma \max_{A'} Q_2[j, k, m']| \\ &\leq \gamma |\max_{A'} Q_1[j, k, m'] - \max_{A'} Q_2[j, k, m']| \end{aligned}$$

$$\leq \gamma \|Q_1 - Q_2\|_\infty \quad (19)$$

When a heuristic term is introduced, it is assumed that the heuristic signal $H[i, j, m]$ is bounded and deterministic. Since $H[i, j, m]$ is not a function of Q , its inclusion results in a shifted Bellman operator but does not affect the contraction property:

$$\begin{aligned} & |\mathcal{T}_H Q_1[i, j, m] - \mathcal{T}_H Q_2[i, j, m]| = \\ & \gamma |\max_{A'} Q_1[j, k, m'] - \max_{A'} Q_2[j, k, m']| \end{aligned} \quad (20)$$

Therefore, the heuristic-augmented Bellman operator also satisfies:

$$\|\mathcal{T}_H Q_1 - \mathcal{T}_H Q_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty \quad (21)$$

By the Banach fixed-point theorem [47], we conclude that repeated application of $\mathcal{T}_H$ converges to a unique fixed point.

Assumptions: We assume (1) the heuristic $H[i, j, m]$ is uniformly bounded, (2) learning rate α_k satisfies standard Robbins-Monro conditions, and (3) the environment is stationary and the state-action space is finite. These ensure that the convergence result holds.

Theorem 1: Convergence of LLM-QL If in the k th iteration, the learning rate α_k satisfies $\sum_{k=1}^{\infty} \alpha_k = \infty$ and $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$, then after an infinite number of iterations, $Q[i, j, m]$ will almost surely converge to the optimal Q-value function $Q^*[i, j, m]$.

Proof 1: In the k th iteration, the update rule is:

$$\begin{aligned} Q_{k+1}[i, j, m] &= Q_k[i, j, m] + \alpha_k \\ & (R[i, j, m] + \gamma \max_{A'} Q_k[j, k, m'] + H[i, j, m] - Q_k[i, j, m]) \end{aligned} \quad (22)$$

Let $\Delta_k[i, j, m] = Q_k[i, j, m] - Q^*[i, j, m]$ denote the error. Then:

$$\begin{aligned} \Delta_{k+1}[i, j, m] &= (1 - \alpha_k) \Delta_k[i, j, m] \\ &+ \alpha_k (\mathcal{T}_H Q_k[i, j, m] - Q^*[i, j, m]) \end{aligned} \quad (23)$$

Since $\mathcal{T}_H$ is a γ -contraction and $\|\mathcal{T}_H Q_k - Q^*\|_\infty \leq \gamma \|\Delta_k\|_\infty$, the Q-update can be regarded as a stochastic approximation process. Under the Robbins-Monro conditions, this guarantees almost sure convergence to the fixed point Q^* .

$$\lim_{k \rightarrow \infty} \|Q_k - Q^*\|_\infty = 0 \quad (24)$$

Theorem 2: Error Bound with Approximate Heuristics Suppose the heuristic approximation error is bounded as $|H[i, j, m] - H^*[i, j, m]| \leq \lambda_H$, where $H^*[i, j, m]$ is the ideal heuristic. Then the steady-state Q-value error is bounded by:

$$\|Q - Q^*\|_\infty \leq \frac{\lambda_H}{1 - \gamma} \quad (25)$$

Proof 2: Substituting the bounded heuristic into the Bellman error:

$$\|Q - Q^*\|_\infty \leq \gamma \|Q - Q^*\|_\infty + \lambda_H \quad (26)$$

Solving the inequality gives the final bound.

TABLE III
TABLE OF KEY EXPERIMENTAL PARAMETERS

Parameter	Value
Number of customers in Seattle	8, 10, 25, 50, 100
Number of UAVs per truck	1, 2, 3
Learning rate (α)	0.1
Discount factor (γ)	0.9
Exploration rate (ε)	0.6
Maximum iterations	1000
UAV battery life	15 min
UAV flying speed	31.29 m/s
Truck driving speed	20 m/s
Maximum UAV payload	5 lbs
Version of LLM	ChatGPT-4o

IV. EXPERIMENTS AND ANALYSIS

A. Experimental Design

To validate the performance of LLM-QL in solving mFSTSP, experiments were designed for investigation. The Seattle city dataset [2] was selected for the experiment, which includes the warehouse and customer locations (latitude and longitude) as well as the parcel weight (in pounds) delivered to each customer. To verify the generality of LLM-QL, we generated a synthetic dataset (within a 40×40 mile area) following the parameter settings described in [1]. The distance for truck delivery between any two points is measured as the real road network distance, while the distance for UAV delivery is measured using euclidean distance. It is specified that each UAV needs 30 minutes of charging after landing. The key parameters for the experiment are in Table III.

Although our experiments use ChatGPT-4o as the underlying LLM, the proposed LLM-QL framework is model-agnostic and applicable to other models such as GPT-3.5, Claude, or DeepSeek. The enhancement stems from the LLM's general ability to extract task-relevant features and generate heuristic signals, rather than any specific model architecture. Our theoretical analysis, based on the convergence of the heuristic-augmented Bellman operator, does not rely on the internal structure of any particular LLM. Differences in model heterogeneity and architecture fall outside the scope of our reinforcement learning enhancement theory. ChatGPT-4o was selected merely to demonstrate the enhancement, not as the reason for it.

The proposed **LLM-QL** combines the local exploration advantages of Q-Learning with the global understanding of the environment provided by LLMs. The heuristic information provided by the LLM guides the Q-Learning agent, significantly reducing ineffective exploration and thus accelerating convergence speed. There are three comparative methods:

- 1) **MILP**: Murray et al. [2] represented the mFSTSP as a Mixed Integer Linear Programming (MILP) problem and divided it into three sub-problems, which were solved using heuristic algorithms.
- 2) **2PML**: Arishi et al. [34] proposed a two-phase machine learning method (2PML) that combines constraint clustering and deep reinforcement learning. In the first phase, k-means clustering is applied to customer locations, and in the second phase, deep reinforcement learning is used to find the optimal path within each constraint cluster.

- 3) **MAPPO**: Bi et al. [35] proposed a Multi-Agent Proximal Policy Optimization (MAPPO) approach, using Monte Carlo tree search to decompose the vehicle routing problem into stages, accelerating truck route determination, and applying multi-agent reinforcement learning to solve UAV flight trajectories, while employing regularization techniques to improve learning efficiency and stability.

The experimental metrics include total completion time, which refers to the total time required for the truck with UAVs to complete the task, starting from the warehouse, fulfilling all customer demands, and returning to the warehouse; algorithm runtime, which is the time taken by the algorithm to solve the mFSTSP; and UAV utilization, which is the amount of idle time during the UAV's operation, indicating the efficiency of task assignment and avoiding excessive idleness or unbalanced task distribution. For various problem scales, with customer numbers set at 8, 10, 25, 50, and 100, and truck-UAV combinations set at 1, 2, 3, five experiments were designed: "Ablation Experiment" "Comparison of Reward Shaping" "Comparison of Solution Quality vs. Problem Scale" "Comparison of Robustness" "Comparison of Algorithm Runtime vs. Problem Scale", and "Sensitivity Analysis".

B. Ablation Experiment

To validate the guiding effect of the heuristic terms generated by the LLM on the Q-Learning agent, ablation experiments were conducted. The ablation experiments were performed for different problem scales with customer numbers of 10, 25, 50, and 100, comparing the convergence curves of Q-Learning with and without the heuristic guidance from the LLM. The convergence metric used is total completion time.

Fig. 3(a), (b), (c), and (d) present the actual scheduling plans of LLM-QL under the condition of the minimum total completion time for each problem scale in Seattle dataset, and Fig. 3(e), (f), (g), and (h) present path planning results in synthetic dataset. The solid blue lines represent the truck's route, while the dashed lines represent the UAV's flight paths. Red dots denote the warehouse, blue dots represent customers, and the shades of blue indicate the weight of parcels delivered to customers. It is evident that LLM-QL adapts the number of UAVs dispatched based on the customer count. For smaller customer counts, a fewer number of UAVs are deployed to meet the service demands, while for larger-scale scenarios with 50 or 100 customers, 2 or 3 UAVs are dispatched to reduce the truck's detouring demands. Fig. 3(i), (j), (k), and (l) show the convergence curves for the ablation experiments at each problem scale in Seattle and synthetic dataset. It can be observed that for smaller customer numbers, LLM-QL with heuristics, although not always providing the exact optimal total completion time, converges significantly faster than without heuristics. For larger customer numbers, LLM-QL with heuristics converges at a similar speed to the version without heuristics but provides more accurate total completion time solutions. Ablation experiments conducted on two datasets demonstrate the generalizability of LLM-QL.

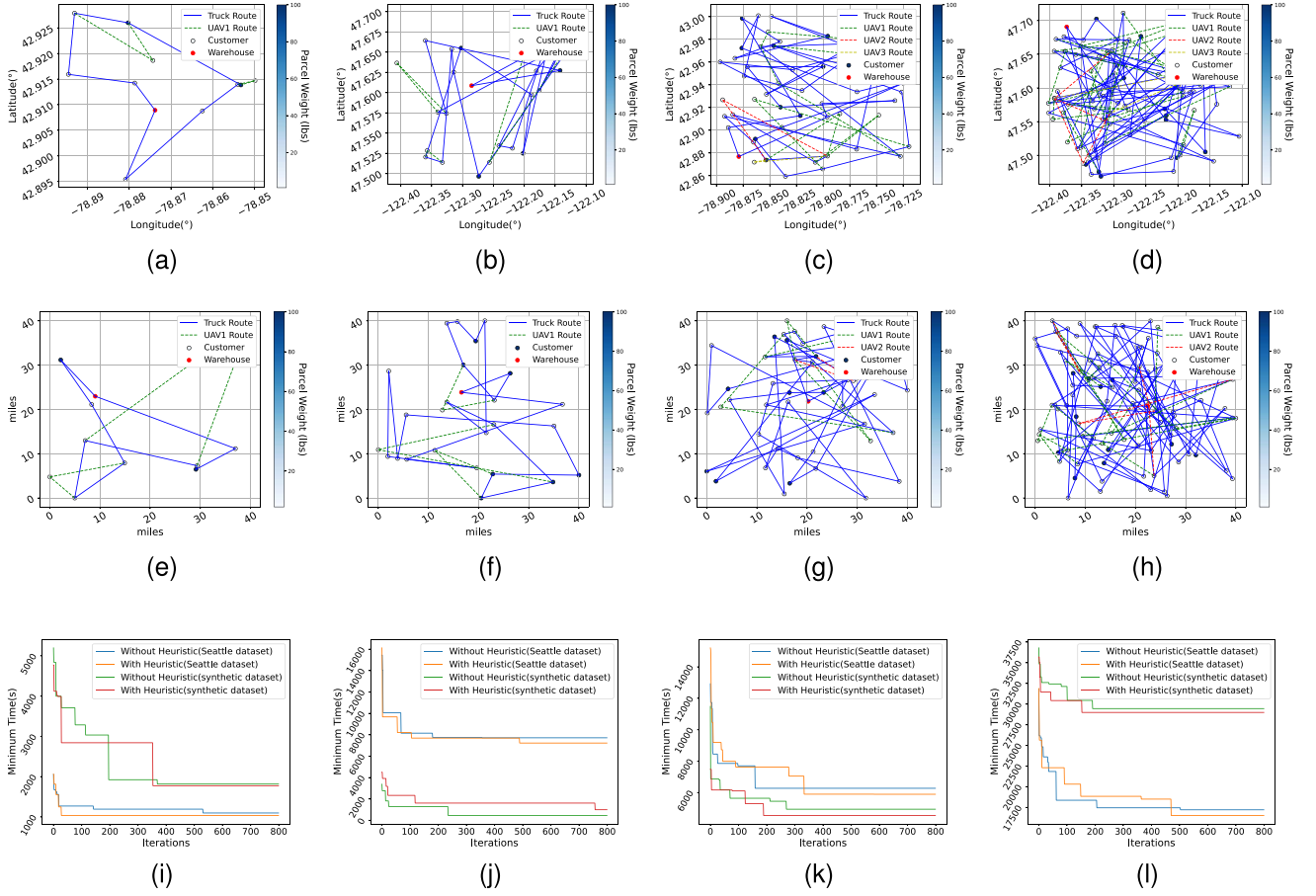


Fig. 3. Path planning results and ablation experiments. (a) Path planning results with 10 customers in Seattle dataset. (b) Path planning results with 25 customers in Seattle dataset. (c) Path planning results with 50 customers in Seattle dataset. (d) Path planning results with 100 customers in Seattle dataset. (e) Path planning results with 10 customers in synthetic dataset. (f) Path planning results with 25 customers in synthetic dataset. (g) Path planning results with 50 customers in synthetic dataset. (h) Path planning results with 100 customers in synthetic dataset. (i) Ablation experiment results with 10 customers. (j) Ablation experiment results with 25 customers. (k) Ablation experiment results with 50 customers. (l) Ablation experiment results with 100 customers.

C. Comparison of Reward Shaping

The experimental results presented in Table IV analyze the cumulative rewards for mFSTSP under various customer sizes (8, 10, 25, 50, and 100 customers), UAV modes (1, 2, 3 UAVs), and iterations (100, 500, and 1000), and reward functions (R_1 , as shown in (13), R_2 , R_3 , as shown in Equation(27)):

$$R_{2,3}[i, j, m] = \begin{cases} \frac{1}{\text{Criterion}[i, j, m]} & \text{If all constraints are satisfied} \\ -\infty & \text{If any constraint is violated} \end{cases} \quad (27)$$

In (13) and (27), R_1 evaluates total travel distance via $T[i, j, m]$, with the reward as its reciprocal if constraints are met, or $-\infty$ otherwise. R_2 and R_3 share the same penalty scheme but differ in objectives: R_2 minimizes service time $\text{Time}[i, j, m]$, and R_3 minimizes energy usage $\text{EnergyUsed}[i, j, m]$, both using reciprocal rewards.

In small-scale settings (8–25 customers), a single UAV often performs best under R_1 due to simpler coordination. For larger scales (50–100), multiple UAVs improve efficiency and rewards. LLM consistently improves performance across all settings by enhancing path planning and reducing redundancy. All reward functions benefit from LLM integration.

Among the three, R_1 consistently yields the best performance, especially with fewer UAVs, as minimizing travel distance is more effective in smaller-scale tasks. R_2 performs reasonably well, particularly as task scale increases and time efficiency becomes critical. R_3 shows weaker performance overall, indicating that energy optimization benefits more from multi-UAV coordination.

D. Comparison of Solution Quality Vs. Problem Scale

From Fig. 4(a), (b), and (c), it can be seen that in experiments assessing total completion time at different problem scales, LLM-QL consistently achieves the lowest total completion time, indicating that LLM-QL can solve for better solutions the fastest within a fixed number of iterations. LLM-QL benefits from the global understanding provided by the LLM to accelerate algorithm convergence. MILP, although finding exact solutions in small-scale problems, experiences exponential growth in computational complexity in large-scale problems, leading to significantly higher total completion times. 2PML requires balancing between clustering and path planning in its two phases, making it difficult to reach optimal solutions in practical applications. MAPPO, while improving efficiency

TABLE IV
REWARD SHAPING

Customer Size	Reward Function	Mode	Iterations								
			100			500			1000		
			1	2	3	1	2	3	1	2	3
8	R_1	With LLM	0.5365	0.512	0.4231	0.6903	0.7211	0.7033	1.0794	0.9450	0.9504
		No LLM	0.3640	0.3534	0.3086	0.6193	0.5479	0.5081	0.9069	0.8269	0.7895
	R_2	With LLM	0.5650	0.4416	0.4162	0.6809	0.6372	0.6236	0.9907	0.9153	0.8120
		No LLM	0.3627	0.2843	0.2632	0.5369	0.5128	0.4269	0.7811	0.8107	0.7346
	R_3	With LLM	0.4536	0.2629	0.3273	0.6899	0.5590	0.4474	0.9243	0.8282	0.8645
		No LLM	0.1506	0.3100	0.1732	0.5273	0.4633	0.3380	0.6591	0.6769	0.7243
10	R_1	With LLM	0.6583	0.6018	0.6048	0.8053	0.7503	0.6213	1.1448	1.0591	0.9829
		No LLM	0.5065	0.4518	0.3697	0.6233	0.5649	0.5118	0.9741	0.8922	0.8059
	R_2	With LLM	0.6419	0.5779	0.5106	0.7852	0.6732	0.6304	1.0866	0.9488	0.9518
		No LLM	0.4113	0.3809	0.3288	0.5847	0.5448	0.5069	0.9449	0.8535	0.8017
	R_3	With LLM	0.3819	0.5253	0.3912	0.6241	0.5226	0.5799	0.9396	0.8805	0.7113
		No LLM	0.4652	0.4161	0.3018	0.5869	0.3282	0.4033	0.6965	0.8473	0.6825
25	R_1	With LLM	0.7374	0.6005	0.5716	0.8759	0.7493	0.7867	1.1634	1.1504	1.1031
		No LLM	0.5351	0.4958	0.4226	0.6565	0.6298	0.5743	0.9721	0.9514	0.8902
	R_2	With LLM	0.6159	0.5226	0.5275	0.8303	0.7712	0.7433	1.1714	1.0148	1.0539
		No LLM	0.5248	0.4246	0.4016	0.6335	0.5562	0.5306	0.9732	0.8683	0.8093
	R_3	With LLM	0.6287	0.5448	0.4166	0.6978	0.5177	0.4613	0.9803	1.0033	0.9664
		No LLM	0.4825	0.2311	0.3068	0.5000	0.5081	0.492	0.8341	0.8608	0.7923
50	R_1	With LLM	0.6152	0.6775	0.7220	0.9928	1.0121	1.0727	1.3565	1.3900	1.4966
		No LLM	0.4712	0.5408	0.6157	0.863	0.8732	0.9278	1.1599	1.2430	1.3229
	R_2	With LLM	0.5683	0.6408	0.725	0.8899	1.0480	1.0394	1.2230	1.3233	1.3539
		No LLM	0.4665	0.5075	0.5368	0.7349	0.8206	0.8686	1.1478	1.1545	1.1889
	R_3	With LLM	0.4685	0.4077	0.5110	0.8217	0.9886	0.9438	1.3084	1.1587	1.2142
		No LLM	0.2835	0.2714	0.5604	0.6782	0.8086	0.7450	0.9646	0.9927	1.1172
100	R_1	With LLM	0.779	0.7766	0.8285	1.5000	1.5708	1.6290	1.7062	1.7198	1.7422
		No LLM	0.5984	0.6445	0.7023	1.2948	1.382	1.4209	1.4674	1.5499	1.6252
	R_2	With LLM	0.6685	0.7315	0.8264	1.4131	1.4874	1.5866	1.5481	1.6277	1.7095
		No LLM	0.4723	0.6063	0.6001	1.2876	1.3039	1.3661	1.5203	1.5176	1.5794
	R_3	With LLM	0.5110	0.7509	0.5768	1.3762	1.3709	1.5039	1.4822	1.4948	1.6459
		No LLM	0.4368	0.3969	0.5233	1.1940	1.2091	1.2651	1.208	1.4535	1.5446

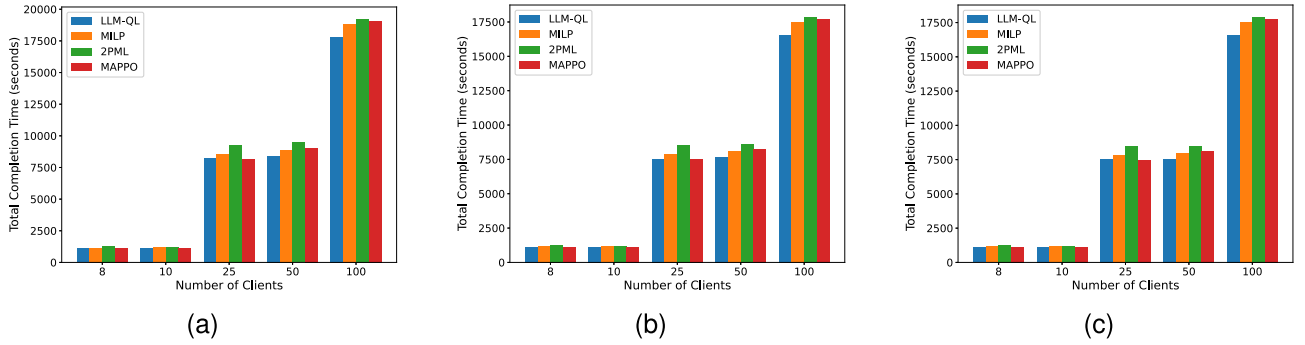


Fig. 4. Total completion time for different problem scales. (a) 1 UAV. (b) 2 UAVs. (c) 3 UAVs.

through multi-agent reinforcement learning, faces performance limitations in large-scale problems due to its algorithmic complexity and computational resource demands. In the large-scale scenario with 100 customers and 3 UAVs, the solution quality of LLM-QL is 1.07 x better than MILP, 1.10 x better than 2PML, and 1.03 x better than MAPPO.

Fig. 5(a), (b), (c), and (d) present a Gantt chart that visually illustrates the mission execution schedule of both the truck and the UAVs. As shown in the chart, the truck is required to remain stationary during UAV launch and recovery operations to ensure safe and stable conditions for takeoff and landing. Likewise, the UAV must wait for the truck to arrive at a pre-designated location before it can initiate the recovery procedure. Once the UAV has successfully launched, the truck is free to proceed with

its own delivery or routing tasks, operating concurrently with the UAVs. Meanwhile, multiple UAVs, once airborne, can coordinate among themselves to serve different customers efficiently, maximizing parallel task execution. This mutual waiting process forms a crucial component of the overall cooperative operational flow. Specifically, during the launch phase, the truck pauses its movement to allow the UAV to take off securely; during the recovery phase, the UAV hovers or waits until the truck reaches the recovery point, enabling a seamless handoff.

From Fig. 6(a), (b), and (c), it can be observed that as the problem scale increases, the UAV utilization of all algorithms increases. Furthermore, for the same algorithm and customer count, higher customer numbers lead to higher UAV utilization. However, LLM-QL consistently maintains a high level of UAV

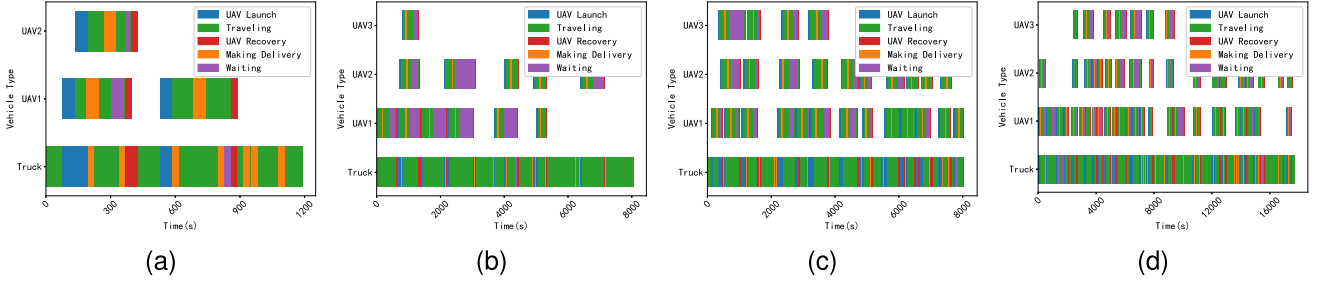


Fig. 5. Gantt chart for truck and UAVs. (a) 10 customers. (b) 25 customers. (c) 50 customers. (d) 100 customers.

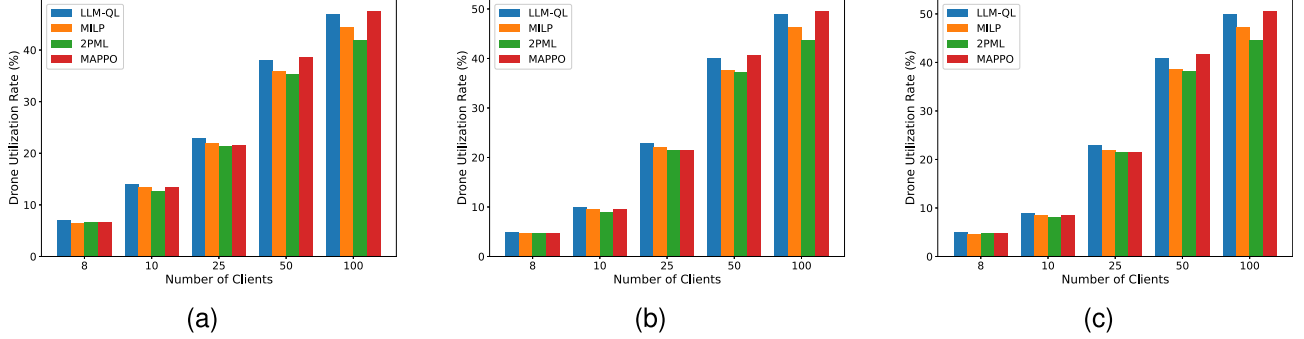


Fig. 6. UAV utilization for different problem scales. (a) 1 UAV. (b) 2 UAVs. (c) 3 UAVs.

utilization, indicating that LLM-QL can adaptively adjust the number of UAVs deployed based on the actual transport situation, avoiding UAV idling or wastage. In contrast, MILP, 2PML, and MAPPO show less favorable performance in UAV utilization. MILP, due to its fixed pattern during the solution process, cannot flexibly adjust UAV usage, resulting in lower utilization. 2PML does not fully account for the actual efficiency of UAV usage during the clustering phase, leading to insufficient UAV utilization in the path planning phase. MAPPO, while improving task assignment flexibility through multi-agent learning, faces limitations in UAV utilization efficiency due to the complexity of its algorithm in large-scale problems. In the large-scale scenario with 100 customers, LLM-QL's UAV utilization is 1.08 x higher than MILP, 1.11 x higher than 2PML, and 1.02 x higher than MAPPO.

E. Comparison of Robustness

As shown in Fig. 7(a), (b), (c), and (d), we analyze the sensitivity of various path planning methods to the number of constraints across different customer sizes. The x -axis represents the number of constraints, ranging from 0 to 11, and the y -axis indicates the performance ratio, which is the current method's performance relative to the best-performing method under all 11 constraints. Across all customer sizes, MILP consistently exhibits the lowest performance, while 2PML and MAPPO perform moderately, and our LLM-QL method achieves the highest performance. As the number of constraints increases from 0 to 11, all methods show improvement, with LLM-QL demonstrating the fastest rate of improvement and reaching 100% performance when all 11 constraints are applied.

This trend can be explained by the characteristics of the methods. MILP, which heavily relies on precise mathematical modeling and optimization, suffers significantly in the absence of constraints, leading to relatively poor performance. In contrast, the 2PML and MAPPO methods, based on reinforcement learning, are more adaptable and less dependent on exact initial modeling. This flexibility allows them to perform better as constraints increase, but they still lack the robustness and performance consistency shown by LLM-QL.

By encoding detailed mathematical models (such as those derived from LaTeX formulas) into the LLM framework, LLM-QL is able to generate precise heuristic guidance even when constraints are incomplete or fluctuating. This allows LLM-QL to efficiently guide the exploration process, ensuring high performance in constraint-rich environments and demonstrating the importance of precise modeling.

Fig. 7(e), (f), (g), and (h) illustrate the convergence behavior and error bound of LLM-QL under different customer sizes, focusing on the effect of heuristic “hallucinations,” or inaccurate/misleading heuristic values generated by LLMs. We simulate this by introducing noise into the heuristics after 500 iterations and observe the system's recovery behavior.

Our results show that while hallucinations temporarily disrupt convergence, LLM-QL consistently recovers and re-converges within a finite number of iterations. In smaller customer scenarios, the search space is simpler and more compact, allowing the model to quickly adjust and return to a near-optimal path. However, in larger environments, the complexity of the problem and expanded decision space lead to slower recovery, as the algorithm must traverse a larger search area to correct the misguidance caused by noisy heuristics.

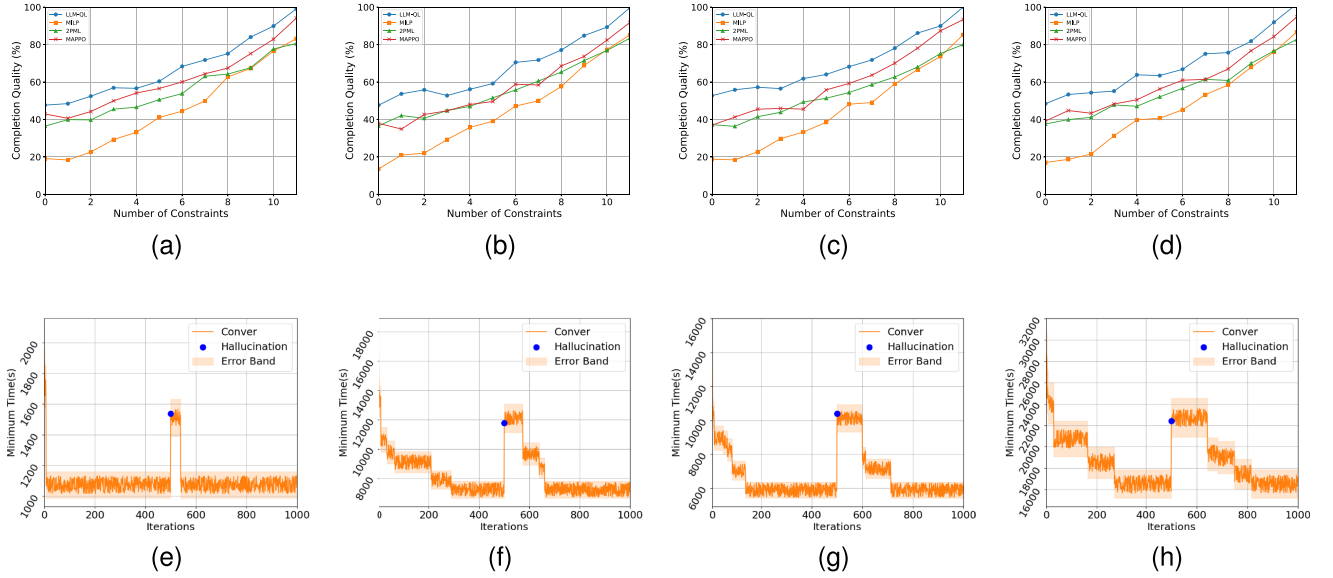


Fig. 7. Robustness experiment results. (a) Sensitivity with 10 customers. (b) Sensitivity with 25 customers. (c) Sensitivity with 50 customers. (d) Sensitivity with 100 customers. (e) Hallucination simulate results with 10 customers. (f) Hallucination simulate results with 25 customers. (g) Hallucination simulate results with 50 customers. (h) Hallucination simulate results with 100 customers.

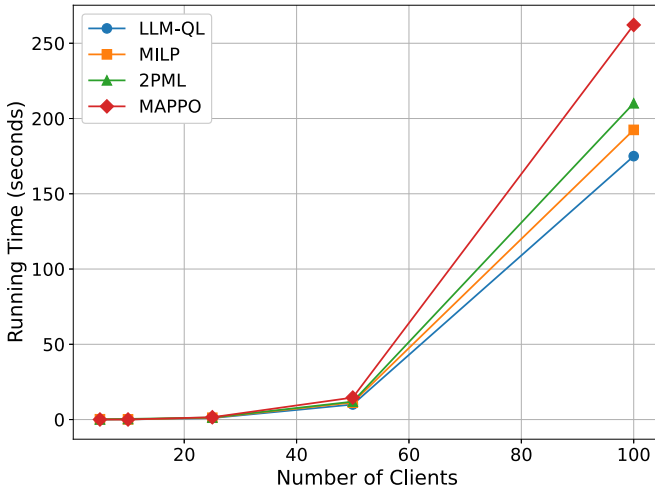


Fig. 8. Algorithm runtime for different problem scales.

Importantly, despite the delays in convergence, the final solution quality remains comparable to that without hallucination. This is because the Q-learning mechanism continues to reinforce better-performing actions over time, gradually diminishing the influence of incorrect heuristic guidance. Although LLM-based heuristics accelerate early exploration, their long-term impact is controlled by the reward-driven learning process. Therefore, even with the presence of heuristic hallucinations, LLM-QL maintains high stability and robustness, ensuring that its solution quality is not significantly compromised in the presence of imperfect or noisy information.

F. Comparison of Algorithm Runtime Vs. Problem Scale

As shown in Fig. 8, all algorithms exhibit exponential runtime growth with problem scale. LLM-QL grows the slowest, as

TABLE V
SENSITIVITY ANALYSIS OF MODEL PARAMETERS

α	γ	ε	Total Completion Time (min)	Algorithm Runtime (s)	UAV Utilization(%)
0.05	0.7	0.4	105.2	150.3	0.85
0.1	0.9	0.6	98.7	145.7	0.88
0.15	0.95	0.8	110.3	160.1	0.82
0.1	0.7	0.6	112.4	148.9	0.83
0.1	0.9	0.4	98.1	141.5	0.89
0.1	0.95	0.6	101.2	155.3	0.86

LLM guidance reduces ineffective exploration. MILP's runtime spikes due to complex integer programming. 2PML narrows the search space but remains costly in deep reinforcement learning. MAPPO improves efficiency via multi-agent learning but struggles with complexity. For 100 customers, LLM-QL is 1.17x, 1.23x, and 1.35x faster than MILP, 2PML, and MAPPO, respectively.

G. Sensitivity Analysis

We conducted a sensitivity analysis to evaluate the impact of key parameters—learning rate (α), discount factor (γ), and exploration rate (ε)—on LLM-QL's performance. By varying each parameter, we observed its effect on total completion time, algorithm runtime, and UAV utilization. The results, shown in Table V, were based on experiments with 50 customers, 3 UAVs, and 1000 iterations.

For the learning rate (α), a moderate value of 0.1 led to faster convergence and lower total completion time, while both very low (0.05) and very high (0.15) values caused slower convergence or oscillation. Regarding the discount factor (γ), a value of 0.9 balanced short-term and long-term rewards effectively, resulting in the best performance. Lowering it to 0.7 led to suboptimal long-term decision-making, while increasing it to

0.95 overemphasized future rewards and worsened performance. For the exploration rate (ϵ), a higher value (0.8) improved UAV utilization through better exploration but increased completion time, while a lower value (0.4) led to faster convergence at the cost of suboptimal task allocation.

V. CONCLUSION

This study proposes a LLM-Enhanced Q-Learning Approach (LLM-QL) to solve the multiple flying sidekicks traveling salesman problem (mFSTSP), which combines the local exploration advantages of Q-Learning with the global reasoning capabilities of large language models, significantly improving the efficiency and effectiveness of solving mFSTSP. Through heuristic guidance, LLM-QL effectively reduces ineffective exploration, leading to accelerated algorithm convergence. Experimental results show that, compared to traditional algorithms, LLM-QL achieves up to a 1.35 x improvement in key performance metrics, demonstrating its superiority in large-scale environments. Future research can focus on further optimizing the real-time performance of LLM-QL and exploring the use of offline, lightweight language models for generating heuristic terms.

REFERENCES

- [1] T. Mulumba and A. Diabat, "Optimization of the drone-assisted pickup and delivery problem," *Transp. Res. Part E: Logistics Transp. Rev.*, vol. 181, 2024, Art. no. 103377.
- [2] C. C. Murray and R. Raj, "The multiple flying sidekicks traveling salesman problem: Parcel delivery with multiple drones," *Transp. Res. Part C: Emerg. Technol.*, vol. 110, pp. 368–398, 2020.
- [3] J. Xu, X. Liu, A. G. Neiat, L. Chu, X. Li, and Y. Yang, "A holistic and hybrid service selection strategy for MEC-based UAV last-mile delivery systems," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3022–3036, Nov./Dec. 2024.
- [4] C. C. Murray and A. G. Chu, "The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery," *Transp. Res. Part C: Emerg. Technol.*, vol. 54, pp. 86–109, 2015.
- [5] P. Bouman, N. Agatz, and M. Schmidt, "Dynamic programming approaches for the traveling salesman problem with drone," *Networks*, vol. 72, no. 4, pp. 528–542, 2018.
- [6] S. T. W. Mara, A. P. Rifai, and B. M. Sopha, "An adaptive large neighborhood search heuristic for the flying sidekick traveling salesman problem with multiple drops," *Expert Syst. Appl.*, vol. 205, 2022, Art. no. 117647.
- [7] N. Agatz, P. Bouman, and M. Schmidt, "Optimization approaches for the traveling salesman problem with drone," *Transp. Sci.*, vol. 52, no. 4, pp. 965–981, 2018.
- [8] A. Rave, "Two-indexed formulation of the traveling salesman problem with multiple drones performing sidekicks and loops," *OR Spectr.*, vol. 47, pp. 67–104, 2024.
- [9] Y. S. Chang and H. J. Lee, "Optimal delivery routing with wider drone-delivery areas along a shorter truck-route," *Expert Syst. Appl.*, vol. 104, pp. 307–317, 2018.
- [10] A. M. Ham, "Integrated scheduling of m -truck, m -drone, and m -depot constrained by time-window, drop-pickup, and m -visit using constraint programming," *Transp. Res. Part C: Emerg. Technol.*, vol. 91, pp. 1–14, 2018.
- [11] M. Dell'Amico, R. Montemanni, and S. Novellani, "Matheuristic algorithms for the parallel drone scheduling traveling salesman problem," *Ann. Operations Res.*, vol. 289, pp. 211–226, 2020.
- [12] Q. Zhou, T. Zhang, J. H. Z. Wu, and H. Dai, "An adaptive path planning algorithm for local delivery of confidential documents based on blockchain," *J. Data Acquisition Process.*, vol. 37, no. 6, 2022, Art. no. 113836.
- [13] Q. Zhou, Z. Sun, J. Wu, H. Dai, and G. Yang, "A location privacy preservation scheme based on consortium block-chain in VANET," *J. Nanjing Univ. Posts Telecommun. (Natural Sci.)*, vol. 42, pp. 85–98, 2022.
- [14] M. Rinaldi, S. Primatesta, M. Bugaj, J. Rostaš, and G. Guglieri, "Development of heuristic approaches for last-mile delivery TSP with a truck and multiple drones," *Drones*, vol. 7, no. 7, 2023, Art. no. 407.
- [15] Q. Zhou, J. Wu, H. Dai, G. Yang, and Y. Zhang, "An intelligent ride-sharing recommendation method based on graph neural network and evolutionary computation," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 1, pp. 569–578, Jan. 2025.
- [16] M. Dell'Amico, R. Montemanni, and S. Novellani, "Drone-assisted deliveries: New formulations for the flying sidekick traveling salesman problem," *Optim. Lett.*, vol. 15, pp. 1617–1648, 2021.
- [17] S. Zhang, H. Zhang, B. Di, and L. Song, "Cellular UAV-to-X communications: Design and optimization for multi-UAV networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 2, pp. 1346–1359, Feb. 2019.
- [18] H. Y. Jeong, B. D. Song, and S. Lee, "Truck-drone hybrid delivery routing: Payload-energy dependency and no-fly zones," *Int. J. Prod. Econ.*, vol. 214, pp. 220–233, 2019.
- [19] R. G. Mbiadou Saleu, L. Deroussi, D. Feillet, N. Grangeon, and A. Quilliot, "An iterative two-step heuristic for the parallel drone scheduling traveling salesman problem," *Networks*, vol. 72, no. 4, pp. 459–474, 2018.
- [20] Y. Chang et al., "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, 2024.
- [21] B. Jin, G. Liu, C. Han, M. Jiang, H. Ji, and J. Han, "Large language models on graphs: A comprehensive survey," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 8622–8642, Dec. 2024.
- [22] Y. Chai et al., "MalFSCIL: A few-shot class-incremental learning approach for malware detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 2999–3014, 2025.
- [23] L. Yang, H. Chen, Z. Li, X. Ding, and X. Wu, "Give us the facts: Enhancing large language models with knowledge graphs for fact-aware language modeling," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3091–3110, Jul. 2024.
- [24] J. Li et al., "Empowering molecule discovery for molecule-caption translation with large language models: A chatgpt perspective," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6071–6083, Nov. 2024.
- [25] Y. Chai, L. Du, J. Qiu, L. Yin, and Z. Tian, "Dynamic prototype network based on sample adaptation for few-shot malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4754–4766, May 2023.
- [26] Q. Zhou, Y. Lian, J. Wu, M. Zhu, H. Wang, and J. Cao, "An optimized Q-learning algorithm for mobile robot local path planning," *Knowl.-Based Syst.*, vol. 286, 2024, Art. no. 111400.
- [27] H. Rong, V. S. Sheng, T. Ma, Y. Zhou, and M. Al-Rodhaan, "A self-play and sentiment-emphasized comment integration framework based on deep Q-learning in a crowdsourcing scenario," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1021–1037, Mar. 2022.
- [28] D. O. Oyewola, S. A. Akinwunmi, and T. O. Omotehinwa, "Deep LSTM and LSTM-Attention Q-learning based reinforcement learning in oil and gas sector prediction," *Knowl.-Based Syst.*, vol. 284, 2024, Art. no. 111290.
- [29] J. Ke, F. Xiao, H. Yang, and J. Ye, "Learning to delay in ride-sourcing systems: A multi-agent deep reinforcement learning framework," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2280–2292, May 2022.
- [30] H. Zhang, Y. Jing, Z. He, K. Zhang, and X. S. Wang, "Learning-based sample tuning for approximate query processing in interactive data exploration," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6532–6546, Nov. 2024.
- [31] X. Wu, "Enhancing Q-learning with large language model heuristics," 2024, arXiv: 2405.03341.
- [32] M. Moshref-Javadi, A. Hemmati, and M. Winkenbach, "A truck and drones model for last-mile delivery: A mathematical model and heuristic approach," *Appl. Math. Modell.*, vol. 80, pp. 290–318, 2020.
- [33] D. Sacramento, D. Pisinger, and S. Ropke, "An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones," *Transp. Res. Part C: Emerg. Technol.*, vol. 102, pp. 289–315, 2019.
- [34] A. Arishi, K. Krishnan, and M. Arishi, "Machine learning approach for truck-drones based last-mile delivery in the ERA of industry 4.0," *Eng. Appl. Artif. Intell.*, vol. 116, 2022, Art. no. 105439.
- [35] Z. Bi, X. Guo, J. Wang, S. Qin, and G. Liu, "Truck-drone delivery optimization based on multi-agent reinforcement learning," *Drones*, vol. 8, no. 1, 2024, Art. no. 27.
- [36] C. Yan and X. Xiang, "A path planning algorithm for UAV based on improved Q-learning," in *Proc. 2nd Int. Conf. Robot. Automat. Sci.*, 2018, pp. 1–5.
- [37] D. Li, W. Yin, W. E. Wong, M. Jian, and M. Chau, "Quality-oriented hybrid path planning based on A* and Q-learning for unmanned aerial vehicle," *IEEE Access*, vol. 10, pp. 7664–7674, 2021.
- [38] J. Wu et al., "An adaptive conversion speed Q-learning algorithm for search and rescue UAV path planning in unknown environments," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 15391–15404, Dec. 2023.

- [39] A. Beishenalieva and S.-J. Yoo, "UAV path planning for data gathering in wireless sensor networks: Spatial and temporal substate-based Q-learning," *IEEE Internet Things J.*, vol. 11, no. 6, pp. 9572–9586, Mar. 2024.
- [40] R. Luo et al., "Valley: Video assistant with large language model enhanced ability," 2023, arXiv: 2306.07207.
- [41] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, "MiniGPT-4: Enhancing vision-language understanding with advanced large language models," 2023, arXiv: 2304.10592.
- [42] M. Trajanoska, R. Stojanov, and D. Trajanov, "Enhancing knowledge graph construction using large language models," 2023, arXiv: 2305.04676.
- [43] S. Meng, Y. Wang, C.-F. Yang, N. Peng, and K.-W. Chang, "LLM-A*: Large language model enhanced incremental heuristic search on path planning," 2024, arXiv: 2407.02511.
- [44] J. Xu, X. Liu, J. Jin, W. Pan, X. Li, and Y. Yang, "Holistic service provisioning in a UAV-UGV integrated network for last-mile delivery," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 380–393, Feb. 2025.
- [45] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
- [46] K. Etessami, A. Stewart, and M. Yannakakis, "Polynomial time algorithms for branching Markov decision processes and probabilistic min (max) polynomial bellman equations," in *Proc. Automata, Lang. Program.: 39th Int. Colloq., ICALP 2012, Warwick, UK, July 9–13, 2012, Part I* 39, 2012, pp. 314–326.
- [47] J. Jachymski, I. Jóźwik, and M. Terepeta, "The banach fixed point theorem: Selected topics from its hundred-year history," *Revista de la Real Academia de Ciencias Exactas, Físicas y Naturales. Serie A. Matemáticas*, vol. 118, no. 4, 2024, Art. no. 140.



Qian Zhou (Member, IEEE) received the PhD degree in computer application technology from the Nanjing University of Aeronautics and Astronautics (NUAA), China, in 2018. She is currently an associate professor with the Nanjing University of Posts and Telecommunications (NJUPT), China, where she also serves as the director of the System Security and Availability Engineering (SAVAGE) Application Technology Institute. Her research interests encompass system security and availability, applied cryptography, network security and privacy, Internet of Things (IoT)

technologies, as well as database systems and other foundational theories and applications.



Jiayang Wu received the BE degree from the Nanjing University of Posts and Telecommunications (NJUPT), China, in 2023. He is currently working toward the ME degree with the same institution. His research interests include network security and privacy, with a specific focus on AI and blockchain technology.



Mengyue Zhu received the BE degree from the Zhengzhou University of Light Industry (ZZULI), China, in 2023. She is currently working toward the ME degree with the Nanjing University of Posts and Telecommunications (NJUPT), China. Her research interests include network security and privacy, with a particular focus on AI and image steganography.



Yuhang Zhou received the BE degree from the Nanjing University of Posts and Telecommunications (NJUPT), China, in 2024. He is currently working toward the ME degree with the same institution. His research interests include network security and privacy, as well as path planning and multi-objective optimization.



Fu Xiao (Senior Member, IEEE) received the PhD degree in computer science and technology from the Nanjing University of Science and Technology, Nanjing, China, in 2007. He is currently a professor and the PhD Supervisor in the School of Computer, Nanjing University of Posts and Telecommunications, Nanjing, China. He has published more than 20 papers in related international conferences and journals, including *IEEE/ACM Transactions on Networking*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Vehicular Technology*, *INFOCOM*, *IPCCC*, *ICC*, and so on. His main research interest include the wireless sensor networks and Internet of Things. He is a member of the *IEEE Computer Society* and the *Association for Computing Machinery*.



Yanchun Zhang (Member, IEEE) received the PhD degree in computer science from the University of Queensland, Gatton, QLD, Australia, in 1991. He is currently a distinguished professor with Zhejiang Normal University, China and Emeritus professor with Victoria University, Australia. He is foreign academician of the Russian Academy of Natural Sciences (RANS), and fellow of Royal Society of Medicine of United kingdom (FRSM). He is the founding director of Centre for Applied Informatics at Victoria University. His research interests include databases, data mining, social networking, web services and e-health/health informatics. He has published more than 400 research papers in international journals and conference proceedings including *ACM Transactions on Computer and Human Interaction (TOCHI)*, *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, *VLDB Journal* and *ICDE* conferences as well as medical journals.